fMRIPrep with Singularity and NVIDIA GPUs

https://gemini.google.com/app/c1c619849a501779

User prompt: Fmriprep, singularity and nvidia

Response: Running **fMRIPrep** within a **Singularity** (Apptainer) container on an **NVIDIA** GPU-enabled HPC environment requires bridging the container runtime with host GPU drivers. While fMRIPrep relies mostly on CPU-heavy operations (e.g., ANTs, FreeSurfer, FSL), specific steps or custom extensions within workflow pipelines benefit from GPU hardware acceleration.

Step 1: Download / Build the Singularity Image (SIF)

Pull the official fMRIPrep Docker image directly into a Singularity SIF file:

```
export SINGULARITY_CACHEDIR="/scratch/$USER/singularity_cache"
export SINGULARITY_TMPDIR="/scratch/$USER/singularity_tmp"

singularity build fmriprep-23.2.0.sif docker://nipreps/fmriprep:23.2.0
```

Step 2: NVIDIA GPU Integration with Singularity

To pass host NVIDIA GPU drivers and CUDA runtimes into the Singularity container, pass the `--nv` flag. This automatically binds `/dev/nvidia*` devices and GPU library paths (`libcuda.so`, `nvidia-smi`).

- **Execution syntax:**

```
singularity run --nv \
    -B /path/to/bids/dataset:/data:ro \
    -B /path/to/output:/out \
    -B /path/to/freesurfer/license.txt:/opt/freesurfer/license.txt \
    fmriprep-23.2.0.sif \
    /data /out participant \
    --participant-label 01 \
    --fs-license-file /opt/freesurfer/license.txt \
    --nthreads 16 \
    --omp-nthreads 8
```

Key Configuration Details

- **The `--nv` Flag:** Ensures host NVIDIA drivers are mounted inside the container runtime without requiring driver installation inside the container image itself.
- **Resource Optimization:** Most fMRIPrep sub-tools (such as `mri_robust_template` or ANTs `antsRegistration`) run on CPUs. Keep allocated CPU thread counts high (`--nthreads`, `--omp-nthreads`) alongside GPU resource requests on Slurm or SGE schedulers.
- **Storage Mounts (`-B`):** Explicitly bind host paths into container mount points for BIDS datasets, output directories, and the required FreeSurfer license file.

Sources

1. How to use a GPU with your Singularity container - HPC discourse
2. GPU Support (NVIDIA CUDA & AMD ROCm) — Singularity
3. Docker Compatibility with Singularity for HPC | NVIDIA Technical Blog
4. Installation — fmriprep version documentation

User prompt: Generate a complete Slurm job submission script for running fMRIPrep with Singularity and NVIDIA GPU support.

Response: Here is a complete Slurm batch submission script (`run_fmriprep.sh`) configured for running fMRIPrep via Singularity with NVIDIA GPU integration:

```
#!/bin/bash
#SBATCH --job-name=fmriprep_gpu
#SBATCH --output=logs/fmriprep_%j.out
#SBATCH --error=logs/fmriprep_%j.err
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=16
#SBATCH --mem=64G
#SBATCH --time=24:00:00
#SBATCH --partition=gpu
#SBATCH --gres=gpu:1

# ------------------------------------------------------------------------------
# 1. Environment & Path Setup
# ------------------------------------------------------------------------------
```

```
# Set directory variables
BIDS_DIR="/path/to/bids_dataset"
OUTPUT_DIR="/path/to/output_directory"
WORK_DIR="/scratch/$USER/fmriprep_work_${SLURM_JOB_ID}"
FS_LICENSE="/path/to/freesurfer/license.txt"
SIF_IMG="/path/to/containers/fmriprep-23.2.0.sif"

# Set cache and temporary directories to scratch space
export SINGULARITY_CACHEDIR="/scratch/$USER/singularity_cache"
export SINGULARITY_TMPDIR="/scratch/$USER/singularity_tmp"

# Create directories if they do not exist
mkdir -p "${OUTPUT_DIR}" "${WORK_DIR}" "logs" "${SINGULARITY_CACHEDIR}" "${SINGULARITY_TMPDIR}"

# Determine participant IDs based on Slurm array or single run
# (If using array jobs, set PARTICIPANT_LABEL=$(printf "%02d" $SLURM_ARRAY_TASK_ID))
PARTICIPANT_LABEL="01"

# ------------------------------------------------------------------------------
# 2. Hardware Verification & Execution
# ------------------------------------------------------------------------------
echo "Job Started on host $(hostname) at $(date)"
echo "Assigned GPU device(s): ${CUDA_VISIBLE_DEVICES}"

# Run Singularity container with NVIDIA GPU flag (--nv)
singularity run --cleanenv --nv \
    -B "${BIDS_DIR}":/bids:ro \
    -B "${OUTPUT_DIR}":/out \
    -B "${WORK_DIR}":/work \
    -B "${FS_LICENSE}":/opt/freesurfer/license.txt:ro \
    "${SIF_IMG}" \
    /bids /out participant \
    --participant-label "${PARTICIPANT_LABEL}" \
    --work-dir /work \
    --fs-license-file /opt/freesurfer/license.txt \
    --nthreads ${SLURM_CPUS_PER_TASK} \
    --omp-nthreads 8 \
    --mem-mb 61440 \
    --output-spaces MNI152NLin2009cAsym:res-2 T1w \
    --skip-bids-validation

# Clean up working directory after successful run
rm -rf "${WORK_DIR}"

echo "Job Finished at $(date)"
```

Key Configuration Details

- **`#SBATCH --gres=gpu:1` & `--nv`:** Allocates one GPU on the Slurm cluster and instructs Singularity to pass the host drivers and libraries inside the container environment.
- **`--cleanenv`:** Prevents environment variables on the host cluster from leaking into the container and causing runtime or Python library conflicts.
- **Working Directory (`--work-dir`):** Directs temporary processing files to rapid scratch storage (`/scratch`) rather than main home directories, preventing disk quota issues.
- **Thread Allocation:** Sets `--nthreads` directly to the allocated Slurm CPU count (`${SLURM_CPUS_PER_TASK}`) and limits OpenMP sub-threads (`--omp-nthreads 8`) to maintain linear scaling across ANTs and FreeSurfer jobs.

User prompt: How do I modify this Slurm script into a job array to process multiple BIDS subjects in parallel?

Response: To convert the script into a Slurm job array, you need to use `#SBATCH --array` and map the `SLURM_ARRAY_TASK_ID` dynamically to your subject IDs.

Here is the updated script configured to process multiple BIDS subjects in parallel:

```
#!/bin/bash
#SBATCH --job-name=fmriprep_array
#SBATCH --output=logs/fmriprep_%A_%a.out
#SBATCH --error=logs/fmriprep_%A_%a.err
#SBATCH --nodes=1
#SBATCH --ntasks=1
```

```
#SBATCH --cpus-per-task=16
#SBATCH --mem=64G
#SBATCH --time=24:00:00
#SBATCH --partition=gpu
#SBATCH --gres=gpu:1
#SBATCH --array=1-10%5  # Run tasks 1 through 10, max 5 running concurrently

# ------------------------------------------------------------------------------
# 1. Environment & Dynamic Subject Mapping
# ------------------------------------------------------------------------------
BIDS_DIR="/path/to/bids_dataset"
OUTPUT_DIR="/path/to/output_directory"
FS_LICENSE="/path/to/freesurfer/license.txt"
SIF_IMG="/path/to/containers/fmriprep-23.2.0.sif"

export SINGULARITY_CACHEDIR="/scratch/$USER/singularity_cache"
export SINGULARITY_TMPDIR="/scratch/$USER/singularity_tmp"

# Option A: Zero-padded numbers (e.g., Task 1 -> sub-01, Task 2 -> sub-02)
PARTICIPANT_LABEL=$(printf "%02d" ${SLURM_ARRAY_TASK_ID})

# Option B (Alternative): Read subject names directly from participant list
# SUBJECTS=($(ls -d ${BIDS_DIR}/sub-* | xargs -n1 basename | sed 's/sub-//'))
# PARTICIPANT_LABEL=${SUBJECTS[$SLURM_ARRAY_TASK_ID - 1]}

# Job-specific working directory to avoid write conflicts during array execution
WORK_DIR="/scratch/$USER/fmriprep_work_sub-${PARTICIPANT_LABEL}_${SLURM_ARRAY_JOB_ID}_${SLURM_ARRAY_TASK_ID}"

mkdir -p "${OUTPUT_DIR}" "${WORK_DIR}" "logs" "${SINGULARITY_CACHEDIR}" "${SINGULARITY_TMPDIR}"

# ------------------------------------------------------------------------------
# 2. Execution
# ------------------------------------------------------------------------------
echo "Processing sub-${PARTICIPANT_LABEL} on host $(hostname) (Array Task ID: ${SLURM_ARRAY_TASK_ID})"

singularity run --cleanenv --nv \
    -B "${BIDS_DIR}":/bids:ro \
    -B "${OUTPUT_DIR}":/out \
    -B "${WORK_DIR}":/work \
    -B "${FS_LICENSE}":/opt/freesurfer/license.txt:ro \
    "${SIF_IMG}" \
    /bids /out participant \
    --participant-label "${PARTICIPANT_LABEL}" \
    --work-dir /work \
    --fs-license-file /opt/freesurfer/license.txt \
    --nthreads ${SLURM_CPUS_PER_TASK} \
    --omp-nthreads 8 \
    --mem-mb 61440 \
    --output-spaces MNI152NLin2009cAsym:res-2 T1w \
    --skip-bids-validation

# Clean up subject-specific working directory
rm -rf "${WORK_DIR}"
```

Key Array Modifications

- **`#SBATCH --array=1-10%5`**: Defines the task IDs. `%5` sets a throttle limit so only 5 jobs run simultaneously, preventing GPU or I/O bottlenecks on shared scratch storage.
- **Log File Formatting (`%A_%a`)**: Sets log filenames to `fmriprep_JOBID_TASKID.out` to isolate outputs for each subject instead of overwriting a single log file.
- **Unique Scratch Directories**: Appends `${SLURM_ARRAY_TASK_ID}` to `WORK_DIR`. If parallel jobs share the same working directory, ANTs and FreeSurfer will corrupt each other's temporary files.
